25MML0014
ANUSREE V
EDGE ASSESEMENT

NETFLIX LOG ANALYSIS

```
import pandas as pd

df = pd.read_json('/content/sample_data/netflix_edge_logs_raw.json', lines=True)
df.head()
```

	timestamp	session_id	edge_server_ip	latency_ms	bitrate_kbps	buffer_events	resolution
0	2026-04-08 12:36:49.466659+00:00	8b0b7f19	10.0.0.6	35	2766	0	1080p
1	2026-04-09 10:37:46.466659+00:00	58ecc91e	10.0.0.6	47	3282	0	480p
2	2026-04-08 22:37:05.466659+00:00	18bd4471	192.168.1.10	45	3715	1	1080p
3	2026-04-09 05:45:01.466659+00:00	87b0d03f	192.168.1.10	29	4720	0	720p
4	2026-04-08 16:57:03.466659+00:00	38310e92	192.168.1.10	52	3648	0	4K

Next steps: Generate code with df | New interactive sheet

Basic Cleaning

[4] ✓ 0s

```
# Check null values
df.isnull().sum()

# Drop duplicates
df = df.drop_duplicates()

# Convert timestamp to datetime
df['timestamp'] = pd.to_datetime(df['timestamp'])
```

```
/tmp/ipykernel_10006/3834878639.py:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexi
  df['timestamp'] = pd.to_datetime(df['timestamp'])
```

Feature Engineering

[6] ✓ 0s

```
# Create buffering flag (target variable)
df['buffer_flag'] = (df['buffer_events'] > 0).astype(int)

# Extract time-based features
df['hour'] = df['timestamp'].dt.hour
df['day'] = df['timestamp'].dt.day
```

Encode Categorical Data

[8] ✓ 0s

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

df['device_type_encoded'] = le.fit_transform(df['device_type'])
```

Outlier Detection & Removal

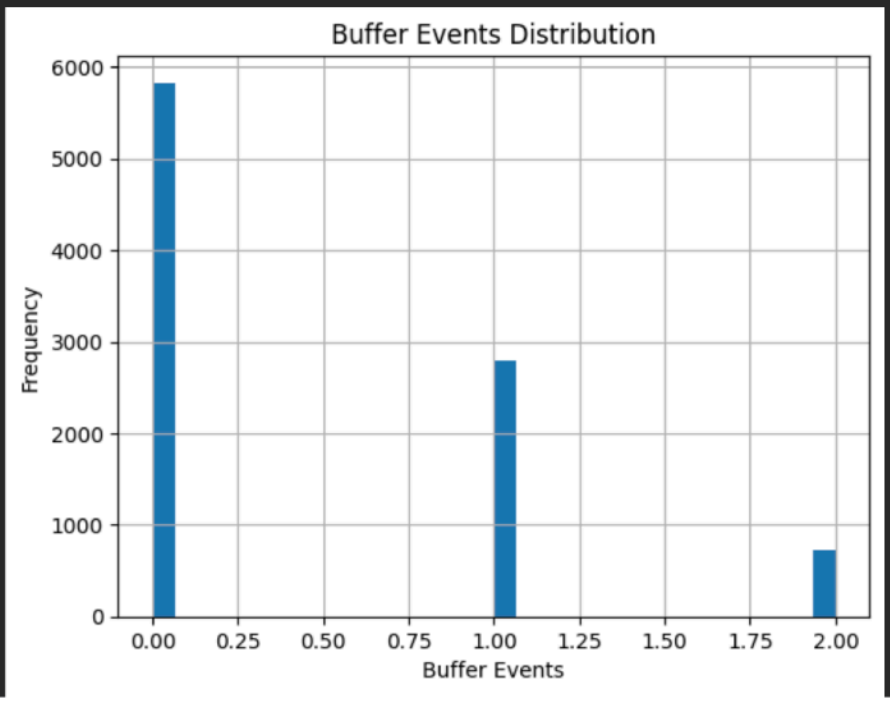
[10] ✓ 0s

```
def remove_outliers(df, column):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    return df[(df[column] >= lower) & (df[column] <= upper)]

# Apply on important columns
df = remove_outliers(df, 'latency_ms')
df = remove_outliers(df, 'buffer_events')
df = remove_outliers(df, 'bitrate_kbps')
```

Data Visualization

2] 2s

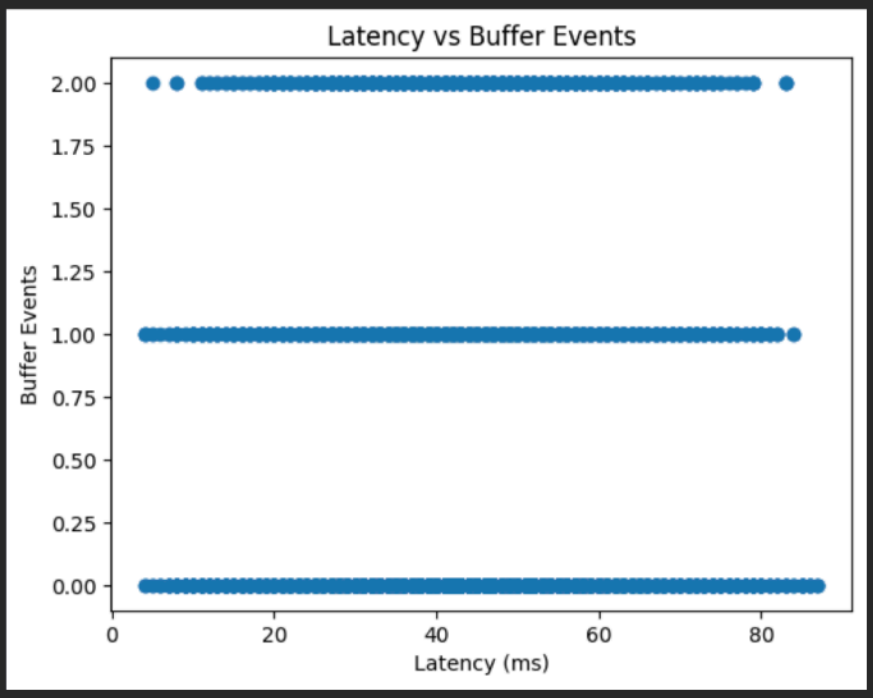
```
import matplotlib.pyplot as plt

df['buffer_events'].hist(bins=30)
plt.title("Buffer Events Distribution")
plt.xlabel("Buffer Events")
plt.ylabel("Frequency")
plt.show()
```



Latency vs Buffering

[14] ✓ 0s

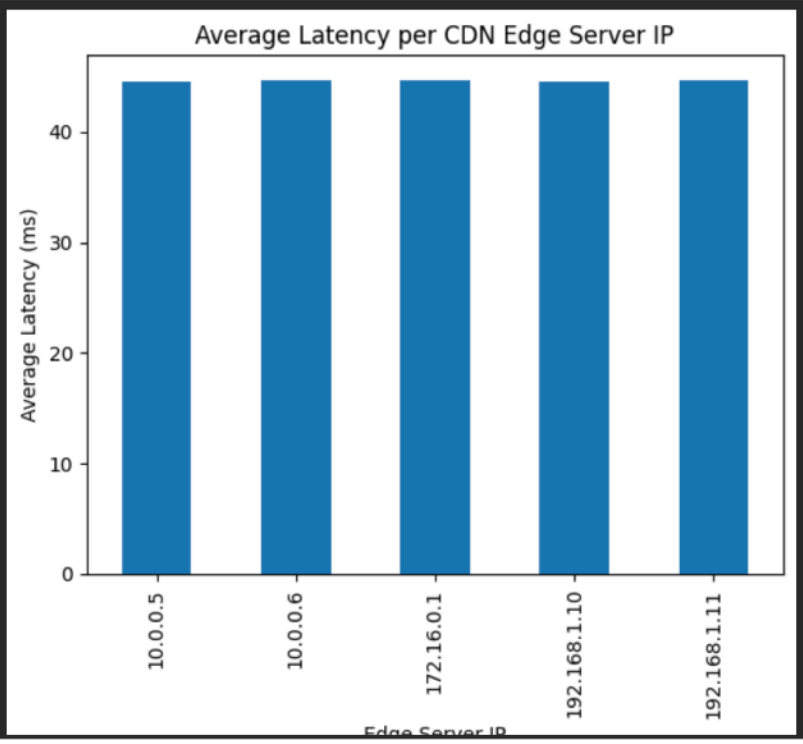
```
plt.scatter(df['latency_ms'], df['buffer_events'])
plt.xlabel("Latency (ms)")
plt.ylabel("Buffer Events")
plt.title("Latency vs Buffer Events")
plt.show()
```



CDN Performance

[16] ✓ 0s

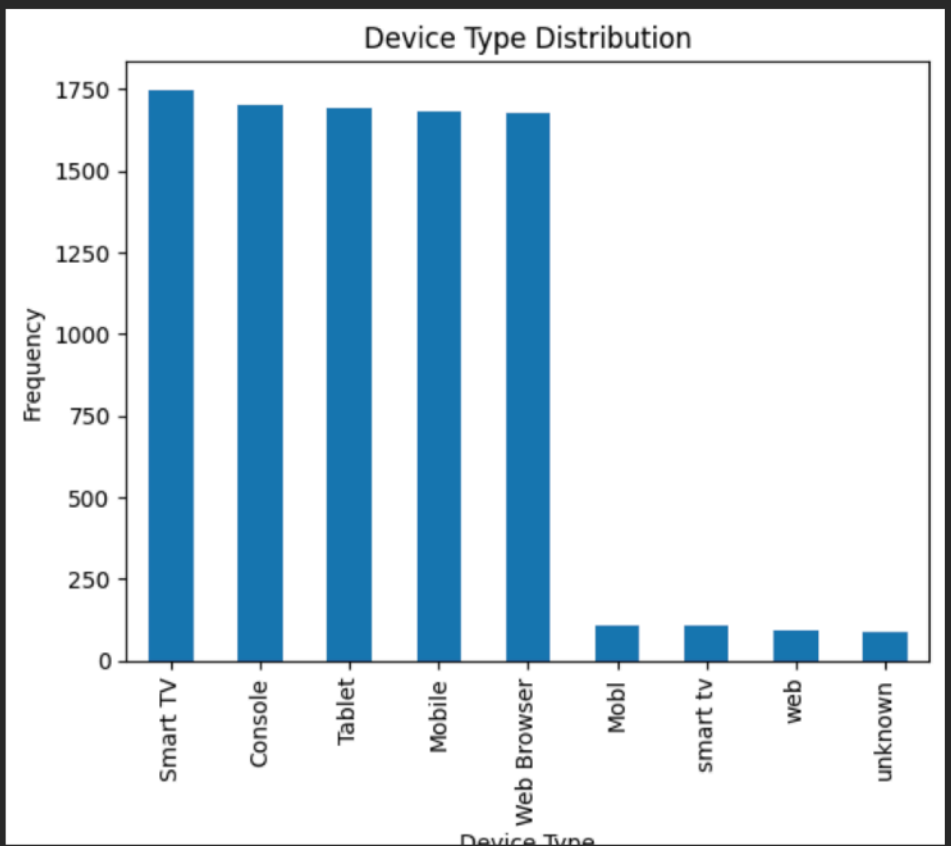
```
df.groupby('edge_server_ip')['latency_ms'].mean().plot(kind='bar')
plt.title("Average Latency per CDN Edge Server IP")
plt.xlabel("Edge Server IP")
plt.ylabel("Average Latency (ms)")
plt.show()
```



Event Distribution

[18] ✓ 0s

```
df['device_type'].value_counts().plot(kind='bar')
plt.title("Device Type Distribution")
plt.xlabel("Device Type")
plt.ylabel("Frequency")
plt.show()
```



Train-Test Split

[20] ✓ 0s

```
from sklearn.model_selection import train_test_split

X = df[['bitrate_kbps', 'latency_ms', 'device_type_encoded', 'hour']]
y = df['buffer_flag']
```

[21] ✓ 0s

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)
```

```
Train shape: (7472, 4)
Test shape: (1869, 4)
```

[22] ✓ 5s

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100)

model.fit(X_train, y_train)
```

```
RandomForestClassifier
nForestClassifier()
```

ode cell output actions

```
from sklearn.metrics import accuracy_score, classification_report

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.5628678437667202
              precision    recall  f1-score   support

           0       0.61      0.80      0.69      1150
           1       0.36      0.18      0.24       719

    accuracy                           0.56      1869
   macro avg       0.49      0.49      0.47      1869
weighted avg       0.51      0.56      0.52      1869
```

```
# Interaction features
df['latency_per_bitrate'] = df['latency_ms'] / (df['bitrate_kbps'] + 1)

# High latency flag
df['high_latency'] = (df['latency_ms'] > 80).astype(int)

# High bitrate flag
df['high_bitrate'] = (df['bitrate_kbps'] > 3000).astype(int)
```

Double-click (or enter) to edit

```
X = df[['bitrate_kbps', 'latency_ms', 'device_type_encoded', 'hour',
        'latency_per_bitrate', 'high_latency', 'high_bitrate']]
```

Hyperparameter Tuning

```
from sklearn.model_selection import GridSearchCV

params = {
    'max_depth': [4, 6, 8],
    'n_estimators': [100, 200],
    'learning_rate': [0.05, 0.1]
}

grid = GridSearchCV(XGBClassifier(), params, cv=3)
grid.fit(X_resampled, y_resampled)

model = grid.best_estimator_
```

Real-Time Processing Simulation

34]
✓ 0s

```python
def process_log(log):
    if log['buffer_time'] > 200:
        return "High Buffering"
    elif log['bitrate'] < 1500:
        return "Low Quality"
    elif log['event'] == 'buffer':
        return "Buffer Event"
    else:
        return "Normal"
```

Apply Processing to All Logs

36]
✓ 0s

```python
# Redefine process_log to use correct column names available in df
def process_log(log):
    if log['bitrate_kbps'] < 1500:
        return "Low Quality"
    elif log['buffer_events'] > 1:
        return "High Buffering"
    elif log['buffer_events'] == 1:
        return "Buffer Event"
    else:
        return "Normal"

df['issue'] = df.apply(process_log, axis=1)
df[['buffer_events', 'bitrate_kbps', 'issue']].head()
```

	buffer_events	bitrate_kbps	issue
0	0	2766	Normal
1	0	3282	Normal
2	1	3715	Buffer Event
3	0	4720	Normal
4	0	3648	Normal

Analytics Engine

[38] ✓ 0s

```
# Average metrics
avg_buffer = df['buffer_events'].mean()
avg_latency = df['latency_ms'].mean()
avg_bitrate = df['bitrate_kbps'].mean()

print("Avg Buffer:", avg_buffer)
print("Avg Latency:", avg_latency)
print("Avg Bitrate:", avg_bitrate)
```

```
Avg Buffer: 0.4534846376190986
Avg Latency: 44.63847553795097
Avg Bitrate: 2992.7548442350926
```

[39] ✓ 0s

```
df['issue'].value_counts()
```

issue	count
Normal	5668
Buffer Event	2713
High Buffering	699
Low Quality	261

dtype: int64

[39] ✓ 0s

```
df['issue'].value_counts()
```

issue	count
Normal	5668
Buffer Event	2713
High Buffering	699
Low Quality	261

dtype: int64

[41] ✓ 0s

```
df.groupby('edge_server_ip')['latency_ms'].mean().sort_values()
```

edge_server_ip	latency_ms
10.0.0.5	44.581206
192.168.1.10	44.600435
192.168.1.11	44.637620
10.0.0.6	44.665067
172.16.0.1	44.705142

dtype: float64

[43]
✓ 0s

```
df.groupby('edge_server_ip')[['buffer_events', 'latency_ms']].mean()
```

edge_server_ip	buffer_events	latency_ms
10.0.0.5	0.460076	44.581206
10.0.0.6	0.459733	44.665067
172.16.0.1	0.439664	44.705142
192.168.1.10	0.476320	44.600435
192.168.1.11	0.432519	44.637620

Decision Engine

[44]
✓ 0s

```
def decision_engine(log):
    if log['buffer_time'] > 200:
        return "Reduce Bitrate"
    elif log['latency'] > 100:
        return "Switch CDN"
    elif log['bitrate'] < 1500:
        return "Increase Bitrate"
    else:
        return "No Action"
```

```
def decision_engine(log):
    if log['buffer_time'] > 200:
        return "Reduce Bitrate"
    elif log['latency'] > 100:
        return "Switch CDN"
    elif log['bitrate'] < 1500:
        return "Increase Bitrate"
    else:
        return "No Action"
```

```
def decision_engine(log):
    if log['buffer_events'] > 1:
        return "Reduce Bitrate"
    elif log['latency_ms'] > 100:
        return "Switch CDN"
    elif log['bitrate_kbps'] < 1500:
        return "Increase Bitrate"
    else:
        return "No Action"

df['action'] = df.apply(decision_engine, axis=1)
df[['buffer_events', 'latency_ms', 'bitrate_kbps', 'action']].head()
```

	buffer_events	latency_ms	bitrate_kbps	action
0	0	35	2766	No Action
1	0	47	3282	No Action
2	1	45	3715	No Action
3	0	29	4720	No Action
4	0	52	3648	No Action

Aggregation

[48] ✓ 0s

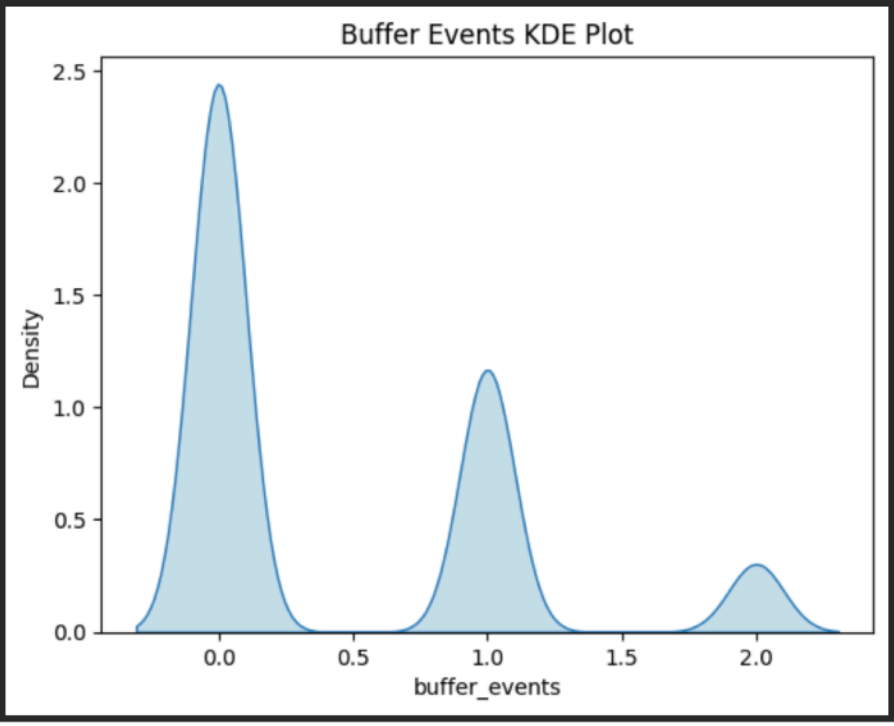
```
# Buffer rate
buffer_rate = df['buffer_flag'].mean()

# Error-like events (buffer events) - using buffer_flag as a direct indicator
error_rate = df['buffer_flag'].mean()

print("Buffer Rate:", buffer_rate)
print("Error Rate:", error_rate)
```

```
Buffer Rate: 0.37608393105663207
Error Rate: 0.37608393105663207
```

KDE Plot

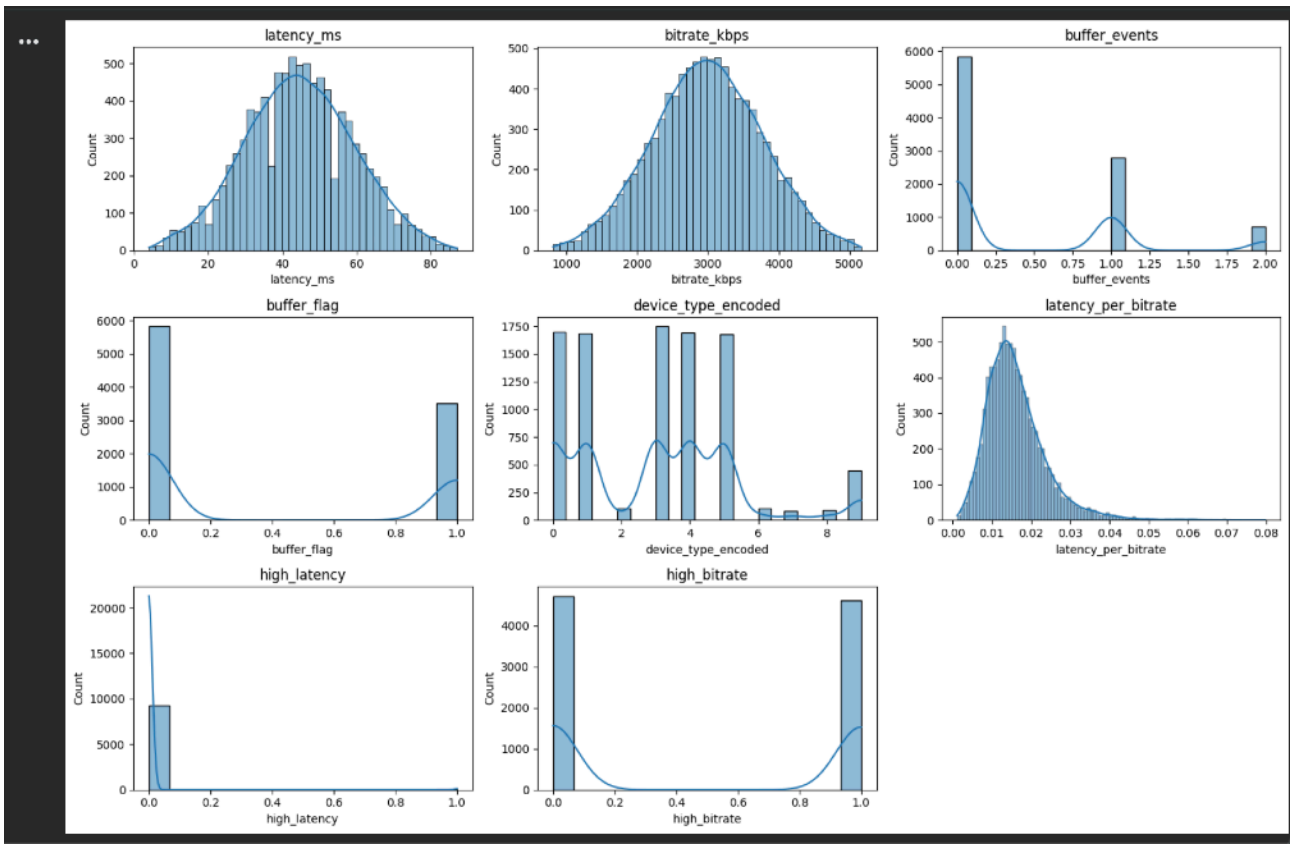
[54] ✓ 0s

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.kdeplot(df['buffer_events'], fill=True)
plt.title("Buffer Events KDE Plot")
plt.show()
```



[57] ✓ 7s

```
import seaborn as sns
import matplotlib.pyplot as plt

num_cols = df.select_dtypes(include=['int64', 'float64']).columns

plt.figure(figsize=(15,10))

for i, col in enumerate(num_cols, 1):
    plt.subplot(3, 3, i)   # adjust rows/cols if needed
    sns.histplot(df[col], kde=True)
    plt.title(col)

plt.tight_layout()
plt.show()
```



Data for Clustering

```
from sklearn.preprocessing import StandardScaler

features = ['bitrate_kbps', 'latency_ms', 'buffer_events', 'device_type_encoded', 'hour']

X = df[features]

# Scale data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

K-Means Clustering

```
from sklearn.cluster import KMeans

k = 3  # Example: choose 3 clusters
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)  # Added n_init to suppress warning
df['cluster'] = kmeans.fit_predict(X_scaled)

print(df['cluster'].value_counts())
df[['bitrate_kbps', 'latency_ms', 'buffer_events', 'cluster']].head()
```

```
cluster
2    3509
0    3040
1    2792
Name: count, dtype: int64
```

	bitrate_kbps	latency_ms	buffer_events	cluster
0	2766	35	0	0
1	3282	47	0	0
2	3715	45	1	2
3	4720	29	0	0
4	3648	52	0	2

```python
from sklearn.manifold import TSNE

tsne = TSNE(n_components=2, random_state=42, perplexity=30)

X_tsne = tsne.fit_transform(X_scaled)
```

```python
import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))

plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=df['cluster'])

plt.title("t-SNE Visualization of Clusters")
plt.xlabel("Component 1")
plt.ylabel("Component 2")
plt.colorbar(label='Cluster')
plt.show()
```